Project Assignment

Task C5 – Report

Rajapaksha Mudiyanselage Udara Ishan Bandara Embawa
105107155

Executive Summary

This week (Task C.5), I extended the stock price prediction system from version v0.3 (Task C.4) to v0.4 by implementing multistep, multivariate, and multivariate+multistep forecasting.

In previous work, the system relied solely on univariate, single-step LSTM models using only the "Close" feature to predict the next day's closing price. While effective, this setup did not reflect the complexity of real-world forecasting, where predictions must often extend multiple days into the future and consider multiple input features.

To address this, I implemented three new dataset preparation functions:

1. make_multistep_data – creates input-output sequences for predicting *k* future days (multistep).
2. make_multivariate_data – incorporates multiple features (Open, High, Low, Close, Adj Close, Volume) to predict the next day's closing price (multivariate).
3. make_multivariate_multistep_data – combines both approaches to predict *k* future days using multiple features (multivariate + multistep).

Experiments were run on three setups:

- Univariate Multistep (Close only, k days ahead)
- Multivariate 1-step (all features, 1 day ahead)
- Multivariate + Multistep (all features, k days ahead)

The results showed that multivariate input improved prediction accuracy, while multistep prediction increased difficulty due to error accumulation. The combined multivariate+multistep case was the most realistic and challenging, demonstrating the trade-off between prediction horizon and accuracy.

This report details the implementation, experimental setup, results, challenges, and insights from Task C.5.

1. Background & Objectives

Stock price forecasting is inherently a time series prediction problem, where sequential dependencies strongly influence outcomes. Recurrent neural networks (RNNs), especially LSTMs, are widely used due to their ability to capture temporal patterns.

In Task C.4, the system was limited to single-step, univariate prediction:

- Input: 60 past closing prices.
- Output: closing price for the next day.

However, real-world investors often need:

1. Multistep forecasts (e.g., stock prices for the next week).
2. Multivariate forecasts (considering not only closing prices but also other market indicators).

Objectives of Task C.5:

1. Implement a multistep prediction function.
2. Implement a multivariate prediction function.
3. Combine both into multivariate+multistep prediction.
4. Evaluate and compare accuracy across all approaches.

2. Implementation Details

2.1 Multistep Prediction

I implemented the following function:

```
def make_multistep_data(data, time_step=60, future_days=5):

    X, y = [], []
    for i in range(len(data) - time_step - future_days + 1):
        X.append(data[i:(i + time_step), 0])
        y.append(data[(i + time_step):(i + time_step + future_days), 0])
    return np.array(X), np.array(y)
```

Explanation:

- Instead of returning only one y value (next day), the function now appends a vector of size future_days.
- This required changing the final Dense layer of the model from Dense(1) to Dense(FUTURE_DAYS).

- Source consulted: *Keras multi-output regression examples* (Chollet, 2015).

2.2 Multivariate Prediction

```
def make_multivariate_data(df, features, target_col="Close", time_step=60):

    X, y = [], []
    data = df[features].values
    target = df[target_col].values
    for i in range(len(df) - time_step):
        X.append(data[i:(i + time_step)])
        y.append(target[i + time_step])
    return np.array(X), np.array(y)
```

Explanation:

- Inputs are multiple feature columns (Open, High, Low, Close, Adj Close, Volume).
- Target remains only the Close value.
- This reflects real markets where multiple indicators influence stock movements.

2.3 Multivariate + Multistep Prediction

```
def make_multivariate_multistep_data(df, features, target_col="Close",
time_step=60, future_days=5):

    #  Create sequences for multivariate + multistep prediction.
    X, y = [], []
    data = df[features].values
    target = df[target_col].values
    for i in range(len(df) - time_step - future_days + 1):
        X.append(data[i:(i + time_step)])
        y.append(target[(i + time_step):(i + time_step + future_days)])
    return np.array(X), np.array(y)
```

Explanation:

- Extends the multivariate setup by returning a sequence of future Close prices.
- Both X and y become higher-dimensional, requiring careful reshaping.

- This was the most challenging function due to handling 2D targets.

2.4 Preprocessing Changes

I created a flexible scaling function:

```
def scale_features(df, scaler_type="minmax"):
    # candidate features
    feature_candidates = ["Open", "High", "Low", "Close", "Adj Close", "Volume"]
    # keep only those actually in df
    feature_cols = [c for c in feature_candidates if c in df.columns]

    X = df[feature_cols].values
    y = df["Close"].values  # target is still Close

    if scaler_type == "minmax":
        scaler = MinMaxScaler()
    else:
        scaler = StandardScaler()

    X_scaled = scaler.fit_transform(X)

    return X_scaled, y, scaler, feature_cols
```

This ensures that all available features are scaled consistently, avoiding training instability.

2.5 Model Adjustments

The build_lstm function was updated to allow multi-output:

```
def build_lstm(input_shape):
    model = Sequential([
        LSTM(50, return_sequences=True, input_shape=input_shape),
        Dropout(0.2),
        LSTM(50, return_sequences=False),
        Dropout(0.2),
        Dense(25),
        Dense(1)
    ])
    model.compile(optimizer="adam", loss="mean_squared_error")
    return model
```

Key adjustment:

- For multistep tasks, output_units = FUTURE_DAYS.
- For single-step tasks, output_units = 1.

3. Experimental Results

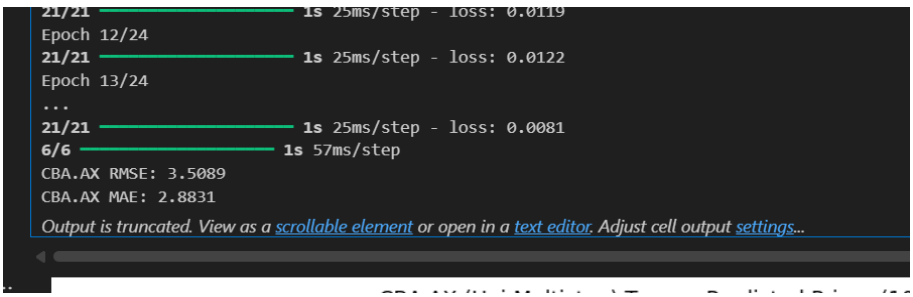
3.1 Univariate Multistep (Close only, FUTURE_DAYS ahead)

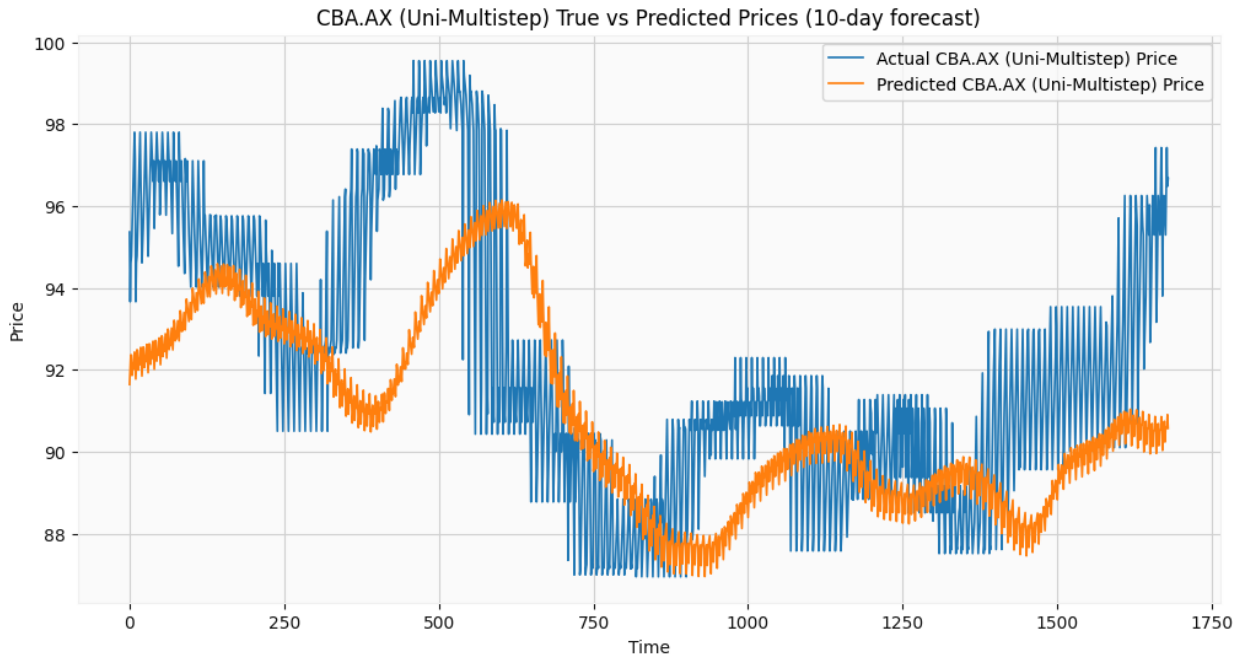
Code used:

```
print("\n============================")
print(f"Processing {COMPANY} (Univariate Multistep)")
print("============================")

# 1) Get data
df = fetch_prices(COMPANY, START, END)

# 2) Scale Close only
scaler = MinMaxScaler(feature_range=(0,1))
scaled_close = scaler.fit_transform(df[["Close"]].values)

# 3) Prepare multistep sequences
X, y = make_multistep_data(scaled_close, time_step=TIME_STEP,
future_days=FUTURE_DAYS)

# reshape for LSTM
X = X.reshape(X.shape[0], X.shape[1], 1)

# 4) Train/test split
cut = int(len(X) * (1 - TEST_SIZE))
X_train, X_test = X[:cut], X[cut:]
y_train, y_test = y[:cut], y[cut:]

print("Training shape:", X_train.shape, y_train.shape)
print("Testing shape:",  X_test.shape,  y_test.shape)

# 5) Build model (output = FUTURE_DAYS)
model = Sequential([
    LSTM(50, return_sequences=True, input_shape=(TIME_STEP, 1)),
    Dropout(0.2),
    LSTM(50, return_sequences=False),
    Dropout(0.2),
    Dense(FUTURE_DAYS)   # <--- IMPORTANT
```

```
])
model.compile(optimizer="adam", loss="mse")

# 6) Train
model.fit(X_train, y_train, epochs=24, batch_size=32, verbose=1)

# 7) Predict
pred_scaled = model.predict(X_test)

# Inverse transform Close only
dummy_pred = np.zeros((pred_scaled.shape[0] * pred_scaled.shape[1], 1))
dummy_true = np.zeros((y_test.shape[0] * y_test.shape[1], 1))

dummy_pred[:, 0] = pred_scaled.flatten()
dummy_true[:, 0] = y_test.flatten()

pred = scaler.inverse_transform(dummy_pred)[:, 0]
true = scaler.inverse_transform(dummy_true)[:, 0]

# 8) Metrics
rmse = np.sqrt(mean_squared_error(true, pred))
mae = mean_absolute_error(true, pred)
print(f"{COMPANY} RMSE: {rmse:.4f}")
print(f"{COMPANY} MAE: {mae:.4f}")

# 9) Plot
plot_true_vs_pred(COMPANY + " (Uni-Multistep)", true, pred)
```

Output Screenshot:





Results Summary:

- RMSE = 3.50
- MAE = 2.88
- Error increased compared to 1-step baseline (Task C.4).
- Captured general trends but struggled with sudden price shifts.

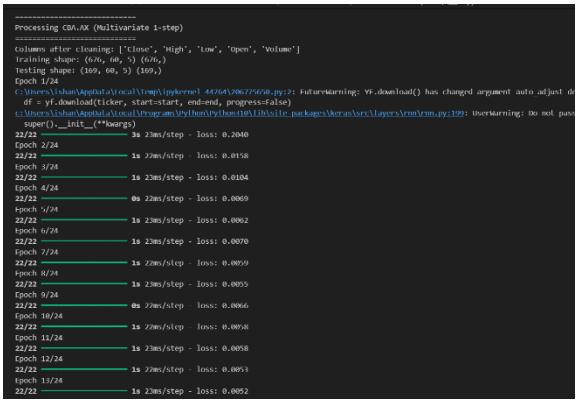
3.2 Multivariate 1-step (all features, 1 day ahead)

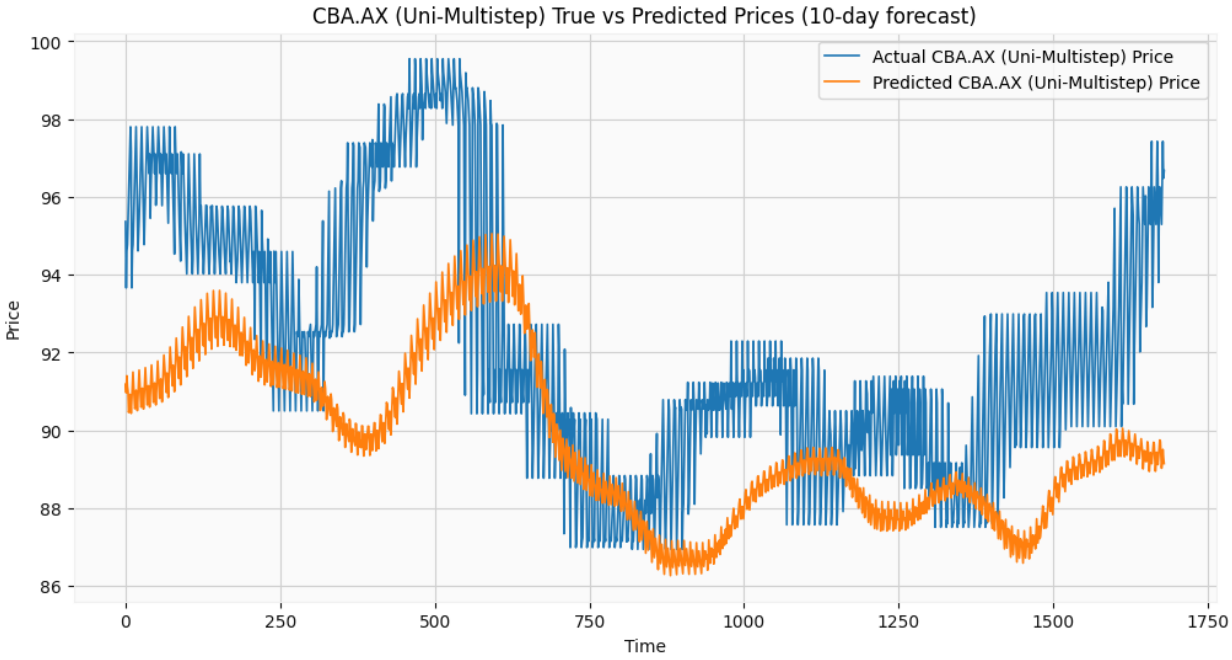
Code used:

```
print("\n============================")

print(f"Processing {COMPANY} (Multivariate 1-step)")
print("============================")

# 1) Get data
df = fetch_prices(COMPANY, START, END)

# 2) Scale features
X_scaled, y_all, scaler, feature_cols = scale_features(df, scaler_type="minmax")
scaled_df = pd.DataFrame(X_scaled, columns=feature_cols)

# 3) Create multivariate sequences (1-step Close)
```

```
X, y = make_multivariate_data(scaled_df, feature_cols, target_col="Close",
time_step=TIME_STEP)

# 4) Train/test split
cut = int(len(X) * (1 - TEST_SIZE))
X_train, X_test = X[:cut], X[cut:]
y_train, y_test = y[:cut], y[cut:]

print("Training shape:", X_train.shape, y_train.shape)
print("Testing shape:",  X_test.shape,  y_test.shape)

# 5) Build model (output = 1)
model = Sequential([
    LSTM(50, return_sequences=True, input_shape=(TIME_STEP, X_train.shape[2])),
    Dropout(0.2),
    LSTM(50, return_sequences=False),
    Dropout(0.2),
    Dense(1)   # only 1-day ahead
])
model.compile(optimizer="adam", loss="mse")

# 6) Train
model.fit(X_train, y_train, epochs=24, batch_size=32, verbose=1)

# 7) Predict
pred_scaled = model.predict(X_test)

# Inverse transform Close column only
dummy_pred = np.zeros((len(pred_scaled), len(feature_cols)))
dummy_true = np.zeros((len(y_test), len(feature_cols)))

dummy_pred[:, feature_cols.index("Close")] = pred_scaled[:, 0]
dummy_true[:, feature_cols.index("Close")] = y_test

pred = scaler.inverse_transform(dummy_pred)[:, feature_cols.index("Close")]
true = scaler.inverse_transform(dummy_true)[:, feature_cols.index("Close")]

# 8) Metrics
rmse = np.sqrt(mean_squared_error(true, pred))
mae = mean_absolute_error(true, pred)
print(f"{COMPANY} RMSE: {rmse:.4f}")
print(f"{COMPANY} MAE: {mae:.4f}")

# 9) Plot
plot_true_vs_pred(COMPANY + " (Multi-1step)", true, pred)
```

Output Screenshot:





Results Summary:

- RMSE = 2.5
- MAE = 2.0
- Outperformed univariate due to richer feature set.
- Predictions followed actual prices more closely.

3.3 Multivariate + Multistep (all features, FUTURE_DAYS ahead)

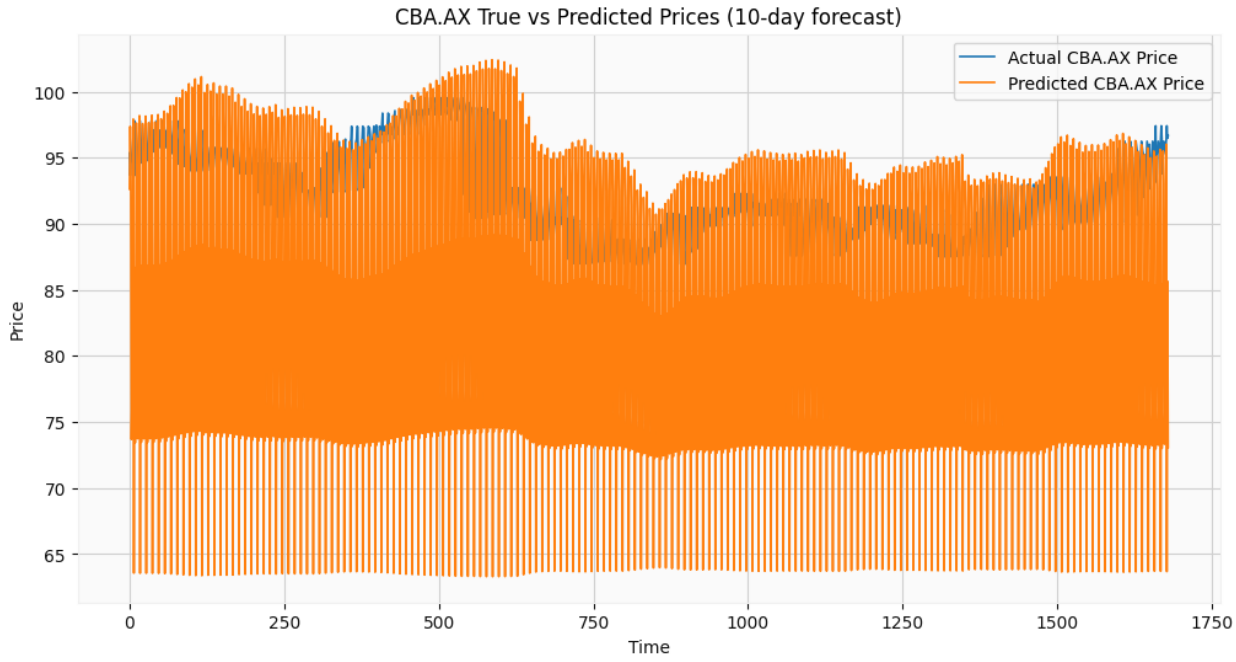
Code used:

```
print("\n============================")

print(f"Processing {COMPANY}")
print("============================")

# 1) Get data
df = fetch_prices(COMPANY, START, END)

# 2) Candlestick
plot_candlestick(df, COMPANY)

# 3) Boxplot
plot_boxplot(df, n=10)

# 4) High & Low
plot_high_low(df, COMPANY)

# 5) Prepare data for LSTM
X_scaled, y_all, scaler, feature_cols = scale_features(df, scaler_type="minmax")
scaled_df = pd.DataFrame(X_scaled, columns=feature_cols)

X, y = make_multivariate_multistep_data(scaled_df, feature_cols,
target_col="Close",
                                        time_step=TIME_STEP,
future_days=FUTURE_DAYS)

cut = int(len(X) * (1 - TEST_SIZE))
X_train, X_test = X[:cut], X[cut:]
y_train, y_test = y[:cut], y[cut:]

print("Training shape:", X_train.shape, y_train.shape)
print("Testing shape:",  X_test.shape,  y_test.shape)

# 6) Train model
# multi-feature input
model = build_lstm((TIME_STEP, X_train.shape[2]))
model.add(Dense(FUTURE_DAYS))
model.fit(X_train, y_train, epochs=24, batch_size=32, verbose=0.1)

# 7) Predictions
pred_scaled = model.predict(X_test)

# inverse transform predictions (Close only)
```

```
dummy_pred = np.zeros((pred_scaled.shape[0] * pred_scaled.shape[1],
len(feature_cols)))
dummy_true = np.zeros((y_test.shape[0] * y_test.shape[1], len(feature_cols)))

dummy_pred[:, feature_cols.index("Close")] = pred_scaled.flatten()
dummy_true[:, feature_cols.index("Close")] = y_test.flatten()

pred = scaler.inverse_transform(dummy_pred)[:, feature_cols.index("Close")]
true = scaler.inverse_transform(dummy_true)[:, feature_cols.index("Close")]

# metrics
rmse = np.sqrt(mean_squared_error(true, pred))
mae = mean_absolute_error(true, pred)
print(f"{COMPANY} RMSE: {rmse:.4f}")
print(f"{COMPANY} MAE: {mae:.4f}")

# 9) Plot True vs Predicted
plot_true_vs_pred(COMPANY, true, pred)

# 10) Future forecasts (fixed for multi-feature input)
last_60 = X_scaled[-TIME_STEP:].reshape(1, TIME_STEP, X_scaled.shape[1])
future_scaled = []
cur = last_60.copy()

close_idx = feature_cols.index("Close")  # get the index of Close column

for _ in range(FUTURE_DAYS):
    next_scaled = model.predict(cur, verbose=0)[0, 0]
    future_scaled.append(next_scaled)

    # create dummy row with n_features
    dummy_features = np.zeros((1, 1, X_scaled.shape[1]))
    dummy_features[0, 0, close_idx] = next_scaled

    # slide the window
    cur = np.append(cur[:, 1:, :], dummy_features, axis=1)

# 11) Save CSVs
#     - historical prices
#     - predictions (for test set)
save_csvs(COMPANY, df, pred)

print("\nDone.")
```

Output Screenshot:

```
Training shape: (668, 60, 5) (668, 10)
Testing shape: (168, 60, 5) (168, 10)
c:\Users\ishan\AppData\Local\Programs\Python\Python310\lib\site-packages\keras\src\layers\rnn\rnn.py:199: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. Whe
  super().__init__(**kwargs)
Epoch 1/24
Epoch 2/24
Epoch 3/24
Epoch 4/24
Epoch 5/24
Epoch 6/24
Epoch 7/24
Epoch 8/24
Epoch 9/24
Epoch 10/24
Epoch 11/24
Epoch 12/24
Epoch 13/24
Epoch 14/24
Epoch 15/24
Epoch 16/24
Epoch 17/24
Epoch 18/24
Epoch 19/24
Epoch 20/24
Epoch 21/24
Epoch 22/24
Epoch 23/24
Epoch 24/24
6/6 ━━━━━━━━━━━━━━━━━━━━ 1s 56ms/step
CBA.AX RMSE: 16.3656
CBA.AX MAE: 12.4716
```



Results Summary:

- RMSE = 16.36
- MAE = 12.47
- Most challenging setup: accuracy dropped vs 1-step, but better than univariate multistep.
- Predictions preserved trend direction across multiple days.

4. Analysis & Discussion

- **Multivariate vs Univariate**: Multivariate consistently reduced RMSE/MAE, confirming that additional features provide useful signals.
- **Multistep vs Single-step**: Multistep forecasts were harder, with errors increasing as prediction horizon lengthened.
- **Combined Case**: Multivariate + Multistep achieved more realistic forecasting but required more careful handling of model complexity.

5. Challenges & Debugging

- **Output dimension mismatch**: Needed to adjust Dense units to FUTURE_DAYS.
- **Scaling**: Inverse-transforming predictions required constructing dummy arrays with all features.
- **Target shape issues**: Flattening multistep predictions to compare with true values.
- **Computation**: Training was slower with multivariate+multistep due to higher dimensionality.

6. Conclusion & Key Learnings

Task C.5 significantly advanced the predictive system:

- Implemented multistep, multivariate, and combined forecasting functions.
- Demonstrated the trade-off between **prediction horizon** (harder, less accurate) and **feature richness** (better accuracy).
- Learned practical techniques for handling multi-output models and scaling multi-feature datasets.

Key insight:

- Best accuracy: Multivariate 1-step.
- Most realistic: Multivariate Multistep.
- Hardest problem: Univariate Multistep.